

Modelling Report

This report documents the modelling work for music genre classification, spectrogram-based similarity search, and genre-based song recommendation. Using spectrogram images and metadata, we developed and evaluated models for automatic genre prediction, visual similarity retrieval, and genre-filtered song suggestions.

Initial situation

The goal of this modelling phase was threefold:

1. Genre Classification: Develop a machine learning model capable of predicting the parent genre of a song based on its spectrogram representation.
2. Similarity Search: Build a feature embedding system to identify visually similar spectrograms for content-based retrieval or music recommendation.
3. Genre-Based Recommendation: Implement a metadata-driven recommender to suggest songs from the same genre, serving as a fast and interpretable baseline.

— This aligns with the *Data Mining Goals* of exploring spectrogram-based learning for music information retrieval —

Data Set(s) and Feature Set

Datasets: Spectrogram images (PNG format) generated from audio tracks, split into: `train`, `val`, `test` folders (64x64 spectrograms).

Label source: `labeled\_song\_artists\_grouped.csv`

15 parent genres defined(Rock, Pop, Hip-Hop & Rap, Electronic, R&B & Soul, Jazz, Classical, Country & Folk, Latin, Metal, Punk & Hardcore, Reggae & Ska, World & International, Blues, Other)

Independent Variables: RGB pixel values of spectrogram images, resized to 224x224 and normalized to [-1, 1].

Target Variable: Parent genre (multi-class classification with 15 categories).

Type of Models Used

MobileNetV3 Large (transfer learning)

- For genre classification
- Library: PyTorch (torchvision)

ResNet-18 feature extractor + Cosine Similarity

- For similarity search
- Library: PyTorch + scikit-learn

Model Descriptions

Genre Classification Model

Overview

- Architecture: MobileNetV3 Large
- Modifications:
 - Replaced classifier head with `Linear(in\_features, 15)`
 - Fine-tuned all layers (no freezing)
- Libraries: torchvision.models, weights=`MobileNet\_V3\_Large\_Weights.DEFAULT`
- Hyperparameters:
 - Batch Size: 32
 - Epochs: 50 (early stopping: patience=5)
 - Learning Rate: 0.0003
 - Optimizer: AdamW
 - Scheduler: CosineAnnealingLR (T_max=50)
 - Loss Function: CrossEntropyLoss (label_smoothing=0.1)
 - Input Image Size: 224x224

Data Augmentation

- Random Horizontal Flip
- Color Jitter (brightness and contrast ± 0.2)
- Normalization: mean=[0.5,0.5,0.5], std=[0.5,0.5,0.5]

Code References

Codebase: ``genre_classifier_mobilenetv3_fixed.pth``

Framework: PyTorch (torch==2.x, torchvision==0.15.x)



Diagram of the genre classification pipline.

Similarity Search Model

Overview

- Architecture: ResNet-18 (final FC layer removed)
- Feature Dimension: 512
- Similarity Metric: Cosine similarity between feature vectors
- Libraries: torchvision.models, weights=``ResNet18_Weights.DEFAULT``

Code References

- Computes embeddings for all spectrograms
- Ranks and retrieves top-4 visually similar spectrograms for a query

Output Artifacts

- ``similar_songs_results/``
- ``query.png`` : Original query spectrogram
- ``visual_1.png`` ... ``visual_4.png`` : Top-4 similar spectrograms
- ``similar_songs.txt`` : List of similar songs with similarity scores
- ``collage.png`` : Collage of query + similar spectrograms



Diagram of the similar songs model pipline.

Genre-Based Recommendation Model

Overview

A simple metadata-based recommendation system that suggests songs from the same genre using filtering. Additionally, a placeholder regression model is trained to simulate feature-based extensions.



Diagram for the genre recommendation medel pipeline.

Language: Python (``pandas``, ``sklearn``)

Steps:

- Encode genre as numeric label
- Fit dummy regression model (non-predictive)
- Filter songs by genre and randomly select from top-N

```
def recommend_by_genre(genre, n_pool=100, top_n=3):
    genre_songs = df[df['genres'] == genre]
    top_n_pool = genre_songs.head(n_pool)
    shuffled = top_n_pool.sample(frac=1).reset_index(drop=True)
    return shuffled[['song_name', 'artist_name', 'genres']].head(top_n)
```

Results

Genre Classification

Metric	Value
Test Accuracy	83.5%
Best Val Accuracy	85.2%
Early Stopping	Triggered at Epoch 37

Confusion Matrix



Training Progress



Plot of “Training and Validation Accuracy over Epochs”



Plot of “Training Loss over Epochs”

Similarity Search

Example Query Results

Rank	Filename	Cosine Similarity
1	Marine_Drive_-_Tuesday_in_Japan_spectrogram.png	0.9810
2	Paranoia_-_Anthony_Rubery_spectrogram.png	0.9809
3	Amnesia_-_SloUCobra_a.k.a._Umberto_spectrogram.png	0.9788
4	LA_Scallywag_-_Hypotenuse_spectrogram.png	0.9785

Collage



Genre-Based Recommender

Sample Output (Genre: Rock)

Song Name	Artist Name	Genre
Backstabber	Abigail Christine Hill	Rock
Tuesday in Japan	Marine Drive	Rock
Paranoia	Anthony Rubery	Rock
Hypotenuse	LA Scallywag	Rock
Don’t Want You Back	Sunbathers	Rock
Metric	Value	
Filtering Speed	Instant (shuffled selection)	
Model Training	Dummy regression (optional)	
Use Case	Cold-start recommender, genre exploration	

Model Interpretation

Classifier

- High accuracy for dominant genres (*Rock*, *Pop*).
- Confusions occur between genres with overlapping spectral features (*Blues* vs *R&B & Soul*).

Similarity Search

- Embeddings visually cluster spectrograms effectively.
- Note: visual similarity ≠ auditory similarity.

Genre-Based Recommendation

- Lightweight, explainable, and effective for cold-start or genre-tagged browsing.
- Easily extendable with learned features or collaborative filtering.

Conclusions and next steps

Key Findings

- The classifier reliably predicts parent genres from spectrograms.
- The similarity model retrieves visually close examples using embeddings.
- Genre filtering offers a strong baseline for interpretable music recommendations.

Limitations

- Genre boundaries are fuzzy and dataset is imbalanced.
- Spectrograms omit rhythm and musical structure.
- Genre-based recommendation is simplistic and non-personalized.

Extensions

- Fuse metadata with embeddings for hybrid recommendations.
- Explore larger architectures (e.g., ViT, EfficientNet).
- Use pretrained audio embeddings (CLAP, OpenL3).
- Integrate user preferences or usage logs for personalization.

Deployment Proposal

- Expose genre classifier as a REST API for tagging tracks.
- Build a discovery UI powered by embedding search.
- Embed genre-based fallback recommender into playlist systems.